

Social Media Data Toolkit: Standardization and pseudonymization of social network datasets

Ali Najafi ^a, Letizia Iannucci ^b, Mikko Kivelä ^b, Onur Varol ^{a,c,*}

^a Faculty of Engineering and Natural Sciences, Sabanci University, Türkiye

^b Department of Computer Science, Aalto University, Finland

^c Center of Excellence in Data Analytics, Sabanci University, Türkiye

ARTICLE INFO

Keywords:

Computational social science
Cross-platform analysis
Data standardization
Data pseudonymization
Social media analytics
Network analysis
Natural Language Processing

ABSTRACT

The rapid diversification of social media platforms and the increasing restrictions on official APIs have significantly complicated cross-platform analysis. Researchers are often forced to rely on heterogeneous datasets obtained through web scraping and historical archives; however they often lack structural consistency. Prior to conducting cross-platform social media analyses, one needs to answer three critical questions: (1) What makes platforms different and similar? (2) How were the datasets collected? (3) How can we align the datasets of different platforms to conduct fair analyses? To address these questions, we introduce the Social Media Data Toolkit (SMDT), a comprehensive Python framework designed for the standardization, pseudonymization, and enrichment of social network datasets. SMDT unifies diverse data structures into a generic schema comprising Communities, Accounts, Posts, Actions, and Entities to facilitate multi-platform research. The framework features a configurable pseudonymization module to secure Personally Identifiable Information (PII) and an extendable enrichment layer that integrates Large Language Models (LLMs) and network analysis tools for downstream tasks such as stance detection and toxicity scoring without creating separate codebases for different datasets. We demonstrate the versatility of SMDT through four case studies spanning from textual analysis of the content to network analysis across platforms. To offer reproducible social media research, SMDT is released as an open-source tool featuring detailed documentation and practical guides for researchers at any skill level. It can be accessed at github.com/ViralLab/SMDT and varollab.com/SMDT.

1. Introduction

As of April 2024, there were 5.44 billion internet users worldwide, or 62.6 percent of the world's population, were on at least one social media platform according to a report by Statista.¹ Given this scale of adoption, social media has become a primary channel for news consumption [1,2], health advice [3,4], financial guidance [5,6], and public discourse. In parallel, regulators have begun to impose accountability on these platforms. The European Union's Digital Services Act (DSA) now subjects very large online platforms (VLOPs) with more than 45 million users to transparency obligations and mandates data access channels for academic researchers. These regulatory shifts create both new opportunities for computational social science and new challenges: researchers must now contend with a far wider variety of data sources and formats than when a single official API sufficed.

The landscape of computational social science has shifted drastically. We have entered a "Post-API" era, where the stable, official

access points that researchers once relied on are disappearing [7]. While major platforms historically offered documented APIs for academic research, the "APIcalypse" saw companies like X (formerly Twitter) and Facebook restrict or monetize access, effectively privatizing public data [8]. This shift poses a severe risk to scientific equity, as high-quality data becomes a luxury available only to well-funded institutions, while others are left to navigate a fragmented landscape of fragile data sources [9].

Facing such constraints, researchers must now piece together datasets from a multitude of acquisition methods: remnant or commercial APIs, web scraping, official data dumps mandated by regulations like the DSA, and voluntary data donations from users. Each method introduces its own structural quirks and limitations. Scraping is sensitive to user interface changes and yields structurally inconsistent data; official data dumps use entirely different, platform-specific schemas; and donated data often requires rigorous pseudonymization before it

* Corresponding author.

E-mail addresses: ali.najafi@sabanciuniv.edu (A. Najafi), letizia.iannucci@aalto.fi (L. Iannucci), mikko.kivela@aalto.fi (M. Kivelä), onur.varol@sabanciuniv.edu (O. Varol).

¹ <https://www.statista.com/statistics/617136/digital-population-worldwide>.

<https://doi.org/10.1016/j.osnem.2026.100356>

Received 1 May 2026; Received in revised form 17 July 2026; Accepted 8 August 2026

Available online 3 September 2026

2468-6964/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1

Feature comparison of social media data toolkits. ✓ = full support, ~ = partial/limited support, · = not supported.

Feature	Tool	SMDT	MADOC [21]	OSoMe [19]	Communalytic [20]	twarc [23]	R SocialMediaLab [24]
Schema unification	✓	~	·	~	·	·	·
Multi-platform	✓	✓	~	✓	·	✓	✓
Live API collection	·	·	✓	✓	✓	·	·
Pseudonymization	✓	~	·	·	·	·	·
GDPR Compliant	✓	~	·	·	·	·	·
Content enrichment	✓	~	~	✓	·	·	·
Network Module	✓	~	~	✓	·	·	✓
Cross-platform query	✓	·	·	·	·	·	·
No-code GUI	·	·	~	✓	·	·	·
Open source	✓	✓	✓	·	✓	✓	✓

can be shared. Consequently, the primary hurdle for modern computational social science is reconciling this heterogeneity of data formats and sources into a cohesive structure suitable for analysis, sharing, and long-term reproducibility [10].

Meanwhile, the social media ecosystem itself continues to fragment. New platforms emerge, some acquiring large user bases while others cater to specific fringe communities [11–15]. Deplatforming events further drive user migration between platforms. When Reddit banned several communities for violating its anti-harassment policy in 2015, and when Twitter suspended former US president Donald J. Trump, users did not simply stop posting—they moved to alternative platforms [16–18]. These migration events make single-platform studies increasingly incomplete, yet multi-platform analyses remain difficult because each platform requires different tools tailored to its specific data format.

Several open-source toolkits have attempted to address parts of this problem. OSoMe, developed by the Indiana University Network Science Institute, provides temporal analysis, network visualization, and geographic mapping, but is designed primarily for Twitter [19]. Communalytic, from the Social Media Lab at Toronto Metropolitan University, offers a no-code interface for data collection across multiple platforms, but is commercially licensed and does not provide a standardized internal data model [20]. MADOC offers pre-standardized datasets for Twitter, BlueSky, Voat, and Koo, but provides only an interface to access existing data rather than a library to transform new data [21,22]. Table 1 compares these and other toolkits across ten dimensions: SMDT offers features for full multi-platform schema unification, pseudonymization, GDPR-compliant erasure, content enrichment, network building, and cross-platform querying within a single open-source framework.

Prior to conducting cross-platform social media analyses, one needs to answer three critical questions: (1) What makes platforms different and what makes them similar? (2) How were the datasets collected? (3) How can we align datasets from different platforms to conduct comparable analyses? Although definitions of concepts and functionalities may differ between platforms, each consists of core abstracted units such as accounts and content. The mechanisms that govern interactions are rooted in the same principles and can be standardized. More importantly, the problems observed on these platforms — misinformation, automated activity, coordinated campaigns, and the strategies behind them — are similar across platforms.

In this work, we make the following contributions through the Social Media Data Toolkit:

- **Data standardization.** We developed a framework that processes datasets from different platforms and transforms them into one unified relational format, supporting 11 platforms through 15+ standardizer variants.
- **Pseudonymization of sensitive information.** Distribution of social media data requires pseudonymization of personally identifiable information. Our framework provides a three-layer configurable pipeline — column-level hashing, regex-based redaction,

and optional NLP-based PII detection — with GDPR-compliant erasure support.

- **Extendable modules.** Our codebase allows developers to introduce their own models for account-level or content-level analysis. Extending the framework to new platforms is straightforward thanks to the abstractions provided by SMDT.
- **Documentation and recipes.** We offer reproducible analyses and recipes to conduct analysis with different platforms and case studies at varollab.com/SMDT.

2. Social media platforms and data

The growing number of social media platforms presents significant challenges for developing cross-platform systems, as each service offers unique features and user dynamics. This work aims to develop a unified framework capable of incorporating various datasets and transforming them into a standardized format. To achieve this, as mentioned previously, we need to address the three critical questions which can help us in designing such a system and gives the researchers the idea of how to use such system.

Question 1: What makes platforms different and similar? Social media architectures vary significantly, enforcing distinct interaction patterns. For instance, threaded discussion platforms like Reddit require users to submit content within specific communities (subreddits), utilizing nested comments and scoring systems to organize discourse. In contrast, microblogging platforms such as Twitter (X) and Truth Social prioritize user-centric feeds; they distribute content to followers without rigid categorization, though topics are often aggregated via hashtags. Finally, Instant messaging platforms like Telegram and WhatsApp, although they are primarily designed for direct communication, increasingly facilitate content distribution through public channels and groups where creators broadcast to subscribers.

Question 2: How were the datasets collected? Another critical dimension in building a unified architecture is the methodology of data collection, which generally follows two paths: API Access and Web Scraping. Historically, platforms were more generous, providing API access to allow developers to build applications and researchers to study user behavior. More recently, for platforms that lack official API access, researchers must rely on web scraping to collect information from these services.

Another dimension that we should consider is whether the collected data was a historical data or collected via stream. Data acquisition characteristics also vary significantly depending on the provider. Platforms such as Bluesky offer real-time stream data via “Firehose” APIs. While this data is immediate, it often lacks comprehensive metadata regarding users or the full context of the content. Conversely, historical datasets provide a retrospective view that typically exclude any content deleted prior to the moment of collection and provide metadata reflecting at the time data collection.

Data richness is another critical factor when conducting these analyses. In their survey, Guo et al. [25] curated available datasets for

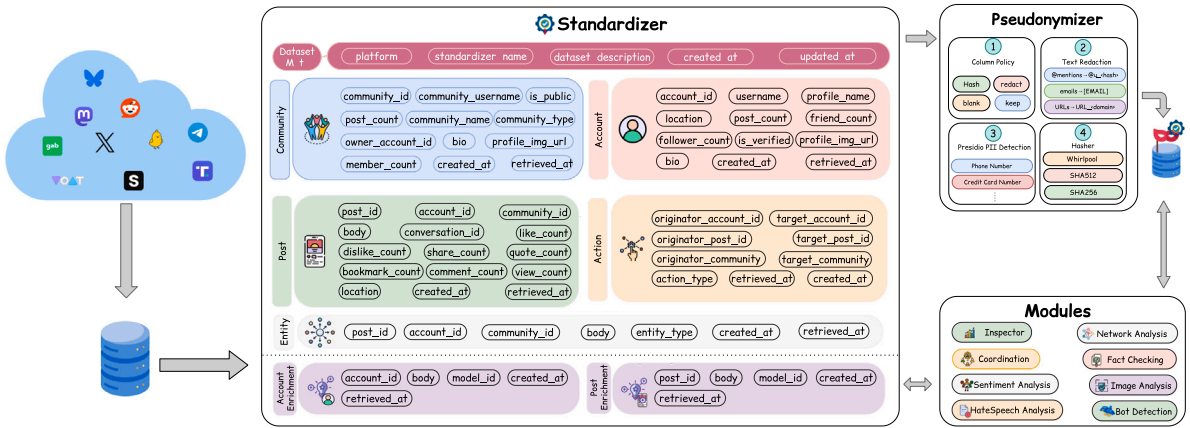


Fig. 1. Schematic of the SMDT platform. The platform consists of distinct modules offering different capabilities. Standardizer unifies data coming from diverse social media platforms into one shared taxonomy. To distribute data for additional analysis, we pseudonymize the sensitive entries and store it further analysis. Framework allows users to use different extensions for enriching dataset by inferring account- and content-level measures. These base information and enrichments can then be used for various down-stream analysis tasks such as coordination detection.

information diffusion tasks. Some datasets are gathered primarily for content-level analysis and lack account-level information, whereas others are compiled specifically for network-level analysis.

Question 3: How can we align the datasets of different platforms to conduct comparable analyses? To answer this core question, we must identify common abstractions for the diverse elements present across different platforms and represent the data within those shared abstractions. Sometimes this alignment is trivial; for instance, a functionality might be called a *retweet* on one platform and a *ReTruth* on another, but both are fundamentally just platform-specific terms for the universal action of *reposting*. In other scenarios, the mapping is significantly more complex, such as attempting to align the nuanced concept of “communities” across platforms that have vastly different underlying interaction mechanics. Furthermore, we must account for the structural inconsistencies and shortcomings stemming from diverse data collection methods. To address these standardization challenges, we introduce SMDT, a framework specifically designed to robustly map heterogeneous platform data into these unified, common abstractions.

3. SMDT for multi-platform analysis

SMDT consists of multiple modules which conforms with the software engineering best practices to provide a flexible, extendable, and scalable framework for conducting social media analyses. Fig. 1 shows the high level structure of our framework. SMDT consists the following components: Standardizer, Inspector, Pseudonymizer, and Enrichers. In the following sections we will describe the functions of each component in details.

3.1. Data standardization

Social media datasets exhibit significant structural heterogeneity. While some platforms provide APIs that segregate objects, such as posts, users, and networks, others stream activities combined with entity metadata. Furthermore, web scraping efforts often result in isolated tabular structures.

The data standardization phase addresses this structural diversity by mapping disparate formats into a unified schema optimized for downstream analysis. Our objective is to populate the fields defined in Table 2 and store the processed data in a PostgreSQL database.

To facilitate access, our framework provides a robust database interface for interacting with these normalized datasets, including support for JSON exports to ensure interoperability with external toolsets. The

proposed generic schema consists of five core tables: Communities, Accounts, Posts, Actions, and Entities, as well as two auxiliary tables: AccountEnrichments and PostEnrichments. Additionally, a Dataset_Meta table keeps metadata information about the ingested dataset. This structure is specifically designed to support both single-platform and cross-platform analysis.

Communities table establishes a foundational taxonomy for digital environments, categorizing spaces according to their underlying communication topologies. We set out a primary distinction between Channels, which are broadcast-oriented, one-to-many architectures optimized for unidirectional information dissemination (e.g., Telegram Channels), and Groups, which act as mesh networks fostering dense, many-to-many discursive interactions. Furthermore, this schema supports hierarchical nesting: Groups may function as autonomous conversation roots (e.g., Reddit submissions, Twitter/X threads) or exist as dependent sub-components embedded within the broader scope of parent Channels.

Accounts table stores user profile metadata functioning as a temporal snapshot, preserving the state of the user profile at the precise moment of data collection. Consequently, there may be a discrepancy between the reported `post_count` attribute in the user’s profile and the actual number of records available in the Posts table, as the latter depends on data sampling and API retrieval limits.

Posts table captures the content generated by the users listed in the Accounts table and functions as the primary persistent store for the content layer of the ecosystem. It keeps the core message body alongside its associated metadata attributes. This schema implements a unified data model to handle heterogeneous content sources, treating distinct entity types, such as a Twitter/X status update or a Reddit platform comment, as structurally equivalent records. A critical architectural detail is the temporal dependency of the metadata: attributes such as `like_count` are recorded as they existed at the specific retrieval timestamp, preserving the engagement state at the point of ingestion rather than maintaining a live synchronization with the source platform.

Actions are introduced to model dynamic interactivity, which functions as a relational record of the edges between Accounts, Posts, and Communities. This schema captures three types of interactions: content engagement (e.g., UPVOTE, SHARE, QUOTE), social ties (e.g., FOLLOW, BLOCK), and structural interlinks (LINK) (see Table S1 for different platforms). This granular classification allows us to filter by `action_type` to construct specific network topologies for downstream analysis. For instance, filtering for SHARE or QUOTE interactions facilitates the construction of information diffusion networks

Table 2

Standardized format of data and its features. There are five main categories for social media data to capture main interaction entities and their actions on social media platforms.

Communities	
community_id	Unique identifier of a community
community_type	Channel or Group
community_username ^a	Username of the community
community_name ^a	Title of the community
bio ^a	Description of the community
is_public ^a	Whether the community is public or not
member_count ^a	Number of accounts in the community
post_count ^a	Number of posts in the community
profile_image_url ^a	Profile Image URL
owner_account_id ^a	Owner Account ID
created_at	Timestamp for the community creation date
retrieved_at	Timestamp for the community collection date
Accounts	
account_id	Unique identifier of an account.
user_name ^a	Unique profile name selected by the user
profile_name ^a	Name selected by account to visualize in the profile
bio ^a	Description
location ^a	Geo Coordinates
post_count ^a	Number of posts linked to account
friend_count ^a	Number of friends account have
follower_count ^a	Number of followers account have
profile_image_url ^a	Profile Image URL
created_at	Timestamp for the account creation date
retrieved_at	Timestamp for the account collection date
Posts	
post_id	Unique identifier of the retrieved post.
account_id	Unique identifier of an account.
conversation_id	Thread's originating post ID.
community_id	Community identifier of the post
body	Information conveyed in the post as a text form
location ^a	Geo Coordinates
like_count ^a	Number of likes
dislike_count ^a	Number of dislikes
view_count ^a	Number of views
share_count ^a	Number of shares
comment_count ^a	Number of replies
quote_count ^a	Number of quotes
bookmark_count ^a	Number of bookmarks
created_at	Timestamp for the post creation date
retrieved_at	Timestamp for the post collection date
Entities	
post_id	ID of a post that contains an entity
body	Information conveyed in a post
entity_type	Type of Entity (Hashtag, Link, etc.)
created_at	Timestamp for the interaction
retrieved_at	Timestamp for the post collection date
Actions	
originator_account_id	ID of an account that initiates interaction
originator_post_id	ID of a post that initiates interaction
target_account_id	ID of an account that was affected
target_post_id	ID of a post that was affected
action_type	Type of interaction like mention, repost, follow, etc.
created_at	Timestamp for the interaction
retrieved_at	Timestamp for the action collection date
AccountEnrichments/PostEnrichments	
account_id/post_id	ID of the account or post
model_id	Object types such as account or post
body	JSON formatted responses of the inference models
created_at	Timestamp for the enrichment creation date
retrieved_at	Timestamp for the enrichment retrieval date

^a Time dependent features captures values at particular timestamp.

^b Static models may increase versions incrementally and dynamics models may link it with timestamp of the model creation.

(similar to retweet graphs), which are instrumental in quantifying political polarization.

A critical architectural detail is the temporal dependency of the metadata: attributes like `like_count` are recorded as they existed at the specific retrieval timestamp, preserving the engagement state at

the point of ingestion rather than maintaining a live synchronization with the source platform. While logging both aggregate metrics in the `Posts` table and discrete interaction events in the `Actions` table may appear redundant from a strict database normalization perspective, this dual structure is necessary due to the inherent limitations of social

media data collection. APIs and web scraping frameworks frequently expose macroscopic engagement metrics (e.g., total shares or likes) without providing access to the complete set of underlying microscopic network edges. Consequently, the quantitative metrics in the `Posts` table serve as an immutable macroscopic snapshot of engagement. In parallel, the `Actions` table logs the specific, granular actions and entities of `Actions` (such as a distinct `SHARE` or `QUOTE` between two identified users) strictly when that explicit relational data is available. This design ensures researchers can analyze total engagement volume even when the complete user-user interaction network cannot be fully reconstructed.

Entities table functions as a granular repository for semantic and structural artifacts extracted from unstructured text fields, such as post bodies, community titles, and user biographies. By systematically indexing auxiliary components, including hashtags, URLs, user mentions, video/image media keys, and email addresses, this table bridges the gap between raw textual content and quantitative analysis. This structure is particularly critical for downstream tasks, such as tracking topic evolution via hashtags and mapping external information flows through URLs.

AccountEnrichments stores supplemental metadata related to user profiles that cannot be directly mapped to the primary account table during standardization. Additionally, it serves as the storage layer for output generated by the various enrichment modules detailed in Section 3.4.

PostEnrichments is similar to the **AccountEnrichments** table. This entity contains post-level metadata that falls outside the standard `Posts` table. It also captures and stores enriched data attributes produced by the post-processing modules. Users can also incorporate their own models to enrich dataset with post- and account-level details.

We implemented each of the aforementioned table schemas using a PostgreSQL backend in a tabular format. The presence of the `created_at` column for all tables is mandatory. Since there can be duplicate records in the collected data, we capture the `retrieved_at` of the records to track temporal changes. The location columns in the `Accounts` and `Posts` tables store geo-coordinates, which can be obtained using the `geopy` library [26] or other alternatives.

For each platform dataset, a dedicated adapter performs the required mapping and passes the generated actions and entities of `Actions` to our database helper. This helper manages bulk insertion and performs on-the-fly deduplication to systematically minimize database collisions. The pipeline uses an “ON CONFLICT DO NOTHING” policy for core tables, which makes repeated ingestion of the same data idempotent and allows interrupted runs to resume safely.

With SMDT, users can concerned with their data mappings and accurate definitions, while the tool manages underlying operations. Listing 1 illustrates the usage of the base `Standardizer` class. To utilize this class, the only required implementation is the `standardize` method, which accepts an `input_record` argument. This argument contains both the raw record and its associated metadata from the source. For example, a record might be a Python dictionary representing a single Twitter data point, while the metadata contains contextual information such as the file path or line number. We have already developed several standardizer adapters for diverse platforms, fully referencing their respective datasets. Furthermore, by extending the base standardizer, researchers can implement custom adapters to utilize SMDT without managing complex database ingestion technicalities. All of this information is available in the software documentation.

3.2. Inspector and dataset comparison

To evaluate our data standardization methodology, we compiled several publicly available social media datasets. First, we examined the primary data entities and their representations across these collections. Table 3 presents the available standardized fields for each platform. A

clear structural distinction emerges between threaded discussion platforms, such as Reddit, and micro blogging platforms like Twitter (X). As discussed for “Communities”, platforms like Reddit and Voat provide granular metadata for sub-communities (e.g., `bio`, `member_count`). In contrast, platforms such as Twitter and Truth Social lack this community layer entirely, organizing content primarily around individual *account ids*.

Furthermore, because some of these datasets were collected via web scraping rather than through official APIs, the absence of certain fundamental data types can be attributed to the methodological limitations inherent to scraping efforts. While the core text content and essential account details are generally obtainable across all platforms, specific temporal metadata is occasionally absent; for instance, datasets from Bluesky, Gab, and Scored lack the `created_at` timestamp for user accounts.

The `Actions` entity remains essential for capturing the diverse array of user interactions, such as sharing, quoting, or replying, which vary significantly in their technical implementation and data availability across these distinct platforms. We do not provide a direct comparison for this entity in the table. Because we aimed to avoid restricting our structure to the limitations of the currently available datasets, we designed the actions schema to comprehensively support the major types of interactions a user can perform on a `Community`, `Account`, or `Post`. Table S1 shows the mapping between each platform’s native actions and our standardized action set. Since data collection and storage schemas (e.g., key names) can differ across datasets, these mappings are defined per platform rather than per dataset.

The SMDT framework operates strictly on an availability basis. It does not impute or synthetically generate missing metadata, such as absent `created_at` timestamps in certain scraped datasets. Downstream modules are designed to process available fields, and we rely on the `Inspector` component to help researchers audit variable availability before designing their cross-platform methodologies.

Conducting cross-platform analyses requires researchers to assess the availability of specific entities within standardized datasets. Previously, we compared this availability across datasets from previously mentioned platforms. However, that reference table may not be applicable when introducing new datasets or alternative platforms. To address this limitation, we introduce an `Inspector` component. Given a database connection (as each dataset is stored in a separate database), this module dynamically calculates the statistics for various entities within each standardized table. Listing 2 provides a code example demonstrating the usage of this module. By instantiating `Inspector` objects for two distinct databases, users can invoke the `report_schemas` function to generate a side-by-side, pretty-printed statistical summary of each table. This functionality allows researchers to quickly observe the available data and determine the types of high-level analyses that are feasible across the selected datasets.

Once data has been ingested into separate databases, users can conduct cross-platform analyses through the `MultiStore` module. Listing 5 shows an example cross-platform query. `MultiStore` attaches multiple standardized databases into a single DuckDB connection and allows users to write SQL joins and unions across them. This works because SMDT intentionally isolates each dataset in its own database while maintaining an identical schema across all of them, so the same column names and types are available regardless of the source platform.

3.3. Data pseudonymization

Social media datasets contain personal identifiers (e.g. account IDs, usernames, profile names) and free-text fields like post bodies and biographies that embed further identifiers such as references to other account names, email addresses, phone numbers, and URLs. Sharing raw datasets across institutions therefore requires transforming these

Standardizer Mapping

```

1  from typing import Any, List, Tuple
2  from smdt.standardizers.base import Standardizer, SourceInfo
3  from smdt.store.models import (Accounts, Posts, Actions,
4      Communities, Entities, PostEnrichments, AccountEnrichments)
5
6  class MyCustomStandardizer(Standardizer):
7      name = "my_custom_standardizer"
8
9      def standardize(
10         self, input_record: Tuple[dict, SourceInfo]) -> List[Any]:
11         record, source_info = input_record
12
13         output_models = []
14         # Users generate model objects and append them to output_models
15         return output_models

```

Listing 1: **Standardizer module.** A sample use case of the Standardizer base class.

Inspector

```

1  from smdt.store.standard_db import StandardDB
2  from smdt.inspector import Inspector, report_schemas
3
4  # 1. Connect to your databases
5  db_name1 = "my_local_db_1"
6  db_name2 = "my_local_db_2"
7
8  sdb1 = StandardDB(db_name1, initialize=False)
9  sdb2 = StandardDB(db_name2, initialize=False)
10
11 # 2. Create an Inspector instance
12 # You can inspect the 'public' schema or any other schema you use
13 inspector1 = Inspector(sdb1, schema="public")
14 inspector2 = Inspector(sdb2, schema="public")
15
16 # 3. Generate and print the report
17 # The report_schemas function handles formatting and printing to stdout
18 # Pass a list of inspectors to compare multiple DBs or schemas if needed
19 report_schemas([inspector1, inspector2], only_tables=["posts", "accounts"])

```

Listing 2: **Inspector module.** Example usage to generate a comparative statistical report across two distinct datasets.

identifiers while preserving analytical utility. We implemented a configurable pseudonymization module that transforms a source database into a structurally identical destination database through a pipeline of column-level and text-level operations. Because a bare hash of a low-entropy identifier, such as a common username or a phone number, remains vulnerable to precomputed dictionary attacks, every hash is keyed with a secret pepper rather than computed directly. The pepper is generated once per deployment and never included in the released dataset. The same keyed hash is applied consistently to every occurrence of an identifier, both in its own column and wherever it appears as a mention within free text, so that interaction networks reconstructed from the pseudonymized data preserve the structure of the original. This design protects against incidental exposure to a party holding only the released dataset and against precomputed dictionary attacks, but not against an adversary who obtains the pepper or who can link a pseudonym to an identity through auxiliary information such as cross-platform behavior or social graph matching. For this reason we describe the module as performing pseudonymization rather than anonymization. Article 4(5) [35] of the GDPR defines pseudonymization as processing personal data such that it can no longer be attributed

to a specific data subject without additional information kept separately, which is precisely the role the pepper plays here. Because that additional information continues to exist and could in principle be used to attribute the data back to an individual, Recital 26 [35] clarifies that pseudonymized data remains information relating to an identifiable natural person, and so remains subject to the same handling obligations as the original.

The transformation applied to each column is governed by a PseudonymPolicy that assigns one of five actions: (i) HASH replaces the value with a deterministic keyed cryptographic hash and is applied to all identifier and foreign-key reference columns; (ii) REDACT applies free-text scrubbing to detect and replace mentions, emails, and URLs, and is applied to post bodies and account biographies; (iii) BLANK sets the column to NULL in the destination, used for fields such as profile image URLs that carry identifying information without analytical value; (iv) DROP removes the column entirely from the destination schema; and (v) KEEP, the implicit default, copies the column unchanged for timestamps, counts, platform labels, and other non-identifying metadata. When a column appears in multiple action sets, a fixed priority order resolves the conflict: DROP takes precedence, followed

Table 3

Comparison of platform data fields. We present data fields available for different platforms and public datasets.

	Twitter V1 [27]	Twitter V2 [27]	BlueSky [28]	Gab [29]	VoatCo [30]	Scored [31]	TruthSocial [13]	Koo [12]	Reddit [32]	USC-TruthSocial [14]	USC-X [33]	Telegram [32]	Parler [34]
Communities													
community_id	.	.	.	.	✓	✓	.	.	✓	.	.	✓	.
community_username	.	.	.	.	✓	✓	.	.	✓	.	.	✓	.
community_name	.	.	.	.	✓	✓	.	.	✓	.	.	✓	.
bio	.	.	.	.	✓	✓	.	.	✓	.	.	✓	.
is_public	.	.	.	.	✓	.	.	.	✓	.	.	✓	.
member_count	.	.	.	.	✓	.	.	.	✓	.	.	✓	.
post_count	.	.	.	.	.	.	.	.	✓	.	.	✓	.
profile_image_url	.	.	.	.	.	.	.	.	✓	.	.	.	.
owner_account_id	.	.	.	.	✓	✓	.	.	✓	.	.	✓	.
created_at	.	.	.	.	✓	✓	.	.	✓	.	.	✓	.
retrieved_at	.	.	.	.	✓	✓	.	.	✓	.	.	✓	.
Accounts													
account_id	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
user_name	✓	✓	.	✓	.	.	✓	✓	✓	✓	✓	✓	✓
profile_name	✓	✓	.	✓	.	.	.	.	✓	.	✓	✓	✓
bio	✓	✓	.	.	.	.	.	✓	.	.	✓	.	✓
location	.	.	.	.	.	.	.	.	.	.	.	.	.
post_count	✓	✓	.	✓	.	.	✓	.	.	.	✓	.	✓
friend_count	✓	✓	.	.	.	.	✓	.	.	.	✓	.	✓
follower_count	✓	✓	.	.	.	.	✓	.	.	.	✓	.	✓
is_verified	✓	✓	.	✓	.	.	.	.	.	.	✓	✓	✓
profile_image_url	✓	✓	.	✓	.	.	.	.	.	.	✓	.	.
created_at	✓	✓	.	.	✓	.	✓	✓	✓	✓	✓	✓	✓
retrieved_at	.	✓	.	✓	.	.	✓	.	✓	✓	.	✓	.
Posts													
account_id	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
post_id	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
conversation_id	.	✓	.	✓	.	.	.	.	✓	.	✓	.	.
community_id	.	.	.	.	✓	✓	.	.	✓	.	.	✓	.
body	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
location	✓	✓	.	.	.	.	.	.	.	.	✓	.	.
like_count	✓	✓	.	✓	.	✓	✓	.	✓	✓	✓	.	✓
dislike_count	.	.	.	✓	.	✓	.	.	.	.	.	.	.
view_count	.	✓	.	.	.	.	.	.	.	✓	✓	.	✓
share_count	✓	✓	✓	✓	.	.	✓	.	.	✓	✓	.	✓
comment_count	✓	✓	✓	✓	.	.	✓	.	.	✓	✓	.	✓
quote_count	✓	✓	.	.	.	.	.	.	.	.	✓	.	.
bookmark_count	.	.	.	.	.	.	.	.	.	.	.	.	.
created_at	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
retrieved_at	.	✓	.	✓	.	.	✓	.	✓	✓	.	✓	.

by HASH, then REDACT, then BLANK, with KEEP as the fallback. For example, the communities.bio column is configured for both HASH and REDACT in the default policy; HASH wins, treating the community description as part of the community's identity rather than as free text to be scrubbed, while accounts.bio is configured only for REDACT and undergoes text-level scrubbing instead.

The hashing component produces deterministic pseudonyms using a two-pass keyed construction. Given a cryptographic hash function H (SHA-256 by default, with SHA-512 and Whirlpool² also supported via Python's hashlib library), a first pass computes $d_1 = H(\text{pepper} \parallel 0x00 \parallel \text{data} \parallel 0xFF)$, and a second pass produces the final pseudonym as $H(\text{pepper} \parallel 0x01 \parallel d_1 \parallel 0xEE)$. This construction is functionally equivalent to HMAC: the pepper is incorporated into every hash, so a bare hash of a low-entropy identifier such as a common username cannot be reversed through a precomputed dictionary attack without knowledge of the pepper. The distinct delimiter bytes between passes

prevent length-extension attacks and mix entropy. Inputs are normalized by stripping surrounding whitespace and lowercasing before hashing, ensuring case-insensitive consistency across all occurrences of an identifier. The hasher preserves NULL and empty-string semantics so that the absence of a reference remains distinguishable from a pseudonymized reference in optional foreign-key columns.

Free-text columns are processed through two alternative redaction paths, selected per column at configuration time; the two paths are mutually exclusive and never both operate on the same column. The default path is a dependency-free Redactor that applies regex pattern matching in multiple passes, each operating on the output of the previous. Mentions (e.g., @JohnDoe) are extracted, lowercased, and hashed using the same Hasher instance and pepper as the column-level HASH action, then replaced with @u_ followed by the hash so that a mention in a post body hashes identically to the corresponding accounts.username value. Email addresses are replaced with the opaque placeholder [EMAIL]. URLs have their hostname extracted and normalized (lowercased, stripped of www. prefix, IDNA-encoded) and are replaced with URL_<hostname> if hostname extraction fails, the generic

² <https://github.com/oohlaf/python-whirlpool>.

placeholder [URL] is used Preserving the registrable domain rather than removing the URL entirely retains coarse link-sharing signal without the identifying path. The mention regex enforces a 64-character maximum on the handle portion after the @ sign; a SHA-256 hex digest is 64 characters, so a hashed mention with its u_ prefix totals 66 characters and cannot be re-matched if the redactor were ever applied twice to the same text.

The regex redactor handles structured identifiers but cannot detect unstructured personal information such as phone numbers, credit card numbers, or named persons that may appear in free text. For datasets requiring stronger privacy guarantees, an optional PII detection layer integrates Microsoft Presidio [36], an open-source NLP-based PII detection library. Since this layer depends on a language model and incurs materially higher processing cost, it is disabled by default. When enabled, it replaces the regex redactor for configured columns and runs Presidio's analyzer and pseudonymizer in a single pass over each text value, detecting entity types specified in a PiiPolicy configuration. The default PII policy scans post bodies and biographies for MENTION (hashed), PHONE_NUMBER (replaced with [PHONE_NUMBER]), EMAIL_ADDRESS (replaced with [EMAIL_ADDRESS]), CREDIT_CARD (replaced with [CREDIT_CARD]), PERSON (replaced with [PERSON]), URL (reduced to domain), and HASHTAG (lowercased with the leading # stripped). Presidio's built-in recognizers for GDPR Article 9 [35] special categories (health status, political opinions, religious beliefs) are deliberately excluded from the default configuration. The PiiPolicy mechanism also accommodates downstream modeling requirements that differ from network analysis requirements: social media text classifiers often expect all mentions collapsed to a single generic token such as @USER rather than distinguished per user, and configuring the mention category to use a fixed replacement rather than a hash produces output compatible with such models, at the cost of the per-user linkability that hashing otherwise preserves.

Different social media platforms use different syntax conventions. Twitter-like platforms mark mentions with @handle and hashtags with #tag, while Weibo uses #tag# with both a leading and trailing hash character. The PII engine selects platform-specific mention and hashtag recognizers at redaction time based on the platform column that every row carries after standardization. Platforms without a custom recognizer fall back to the generic Twitter-like patterns.

A critical correctness requirement is that the same real-world entity pseudonymizes to the same value regardless of which column or processing path it passes through. A user JaneSmith with account ID 2244994945 must produce the same pseudonym whether the value appears in accounts.username, posts.account_id, an @JaneSmith mention in a post body, a USER_TAG entity, or an originator reference in the actions table. This is achieved through two mechanisms: the column-level hasher, the regex redactor's mention handler, the entity sanitizer's user-tag handler, and the PII engine's mention handler all share the same Hasher object with the same pepper and normalization function; and every path that transforms a mention strips the leading @, lowercases the handle, and passes it through Hasher.make(), prefixing the result with @u_. Consequently, interaction networks, foreign-key joins, and cross-table queries all survive pseudonymization intact.

The module does not retain a mapping from pseudonyms back to original identifiers, since such a mapping would function as a complete de-anonymization key for the released dataset and would undermine the purpose of releasing pseudonymized rather than raw data. The GDPR does not require an efficient reverse lookup mechanism, only that re-identification remain possible in principle through information kept separately, and this requirement is satisfied through recomputation rather than storage: given a real, known identity and the deployment's pepper, the corresponding pseudonym can be recomputed on demand and located within a released dataset. This recomputation-based design

supports deletion requests directly. As Listing 4 shows, given an identified individual's account identifier, an erasure module removes their data from the source database, the released dataset, or both, through either hard deletion or in-place scrubbing, with the two modes chosen independently for each target database. Deletion removes the account and its posts entirely. Scrubbing retains the row but replaces personal fields (biography, post body text, username, profile fields, and engagement counts) with NULL values. We treat scrubbing as the safer default for an already released dataset, since deleting a post outright can orphan other users' replies and interactions with it, whereas a scrubbed post keeps its identifier and thread position, so a reply remains attached to a still-present, if content-emptied, parent. Because the account_id field on the posts table cannot be NULL, scrubbing a plaintext database instead assigns each erased individual a freshly generated placeholder identifier, unique to that request, applied consistently across their own account and post rows so their contributions remain grouped as originating from a single, now anonymous, author without being derivable back to the original identifier. On an already pseudonymized database this step is unnecessary, since the existing hashed identifier is already opaque and is left unchanged. Interaction records are handled according to the erased individual's role rather than by mode: an action in which they are the originator (a like, comment, or share they issued) reflects their own behavior and is removed outright in either mode, since no other table references an action row and its removal cannot orphan anything; an action in which they are only the target (a like or follow received from another user) is retained, since it documents the other user's behavior rather than the requester's, and only the reference to the requester is cleared. Entities and enrichment records belonging to the erased individual are likewise removed outright in both modes, since they are not referenced elsewhere in the schema and their retention would serve no structural purpose while still containing personal data.

The pseudonymizer operates as a DB-to-DB ETL pipeline. It reads from the source database using server-side cursors with batched fetching to avoid materializing entire tables in memory, transforms each row according to the configured policy, and flushes batches to the destination database through a three-tier insert fallback chain (COPY, then multi-VALUES, then per-row SAVEPOINT) that isolates malformed rows without losing entire batches. For large datasets, a multiprocess mode distributes the CPU-bound transformation work across worker processes, each constructing its own Hasher, Redactor, and optional PiiEngine once at startup. Workers never open database connections: only the main process reads from source and writes to destination.

Listing 3 illustrates a fully configured pseudonymization run that exercises the module's principal extension points. A custom PseudonymPolicy extends the default column-level actions by additionally blanking the location columns on the Accounts and Posts tables, since geocoordinates can be identifying. A PiiPolicy enables Presidio-based detection on post bodies and account biographies, specifying per-entity-type rules: mentions are hashed for consistency with column-level identifiers, while personal names, phone numbers, and credit card numbers are replaced with entity-type placeholders; hashtags are lowercased and stripped of their leading # character. The policy supports callable replacements, shown here for URLs, which a lambda reduces to their registrable domain. A time window restricts processing to the first two weeks of May 2023, allowing incremental pseudonymization of a date range without reprocessing the entire dataset. Four worker processes distribute the CPU-bound transformation work while the main process alone handles database I/O. A spaCy³ [37] English language model is supplied to Presidio for named-entity recognition. Any of these configuration options can be omitted; when none are provided, the module falls back to the dependency-free regex redactor with the default column policy.

³ <https://spacy.io>.

Sample Pseudonymization - Full Configuration

```

1  from smdt.config import PseudonymizationVariables
2  from smdt.pseudonymizer import (
3      Pseudonymizer, PseudonymizeConfig, Algorithm,
4      DEFAULT_POLICY, PseudonymPolicy,
5  )
6  from smdt.pseudonymizer.pii_policy import (
7      PiiPolicy, PiiRule, PiiAction,
8  )
9
10 pseudo_vars = PseudonymizationVariables()
11
12 # -- Tune the default policy: also blank the location columns -----
13 custom_policy = PseudonymPolicy(
14     hash_cols=DEFAULT_POLICY.hash_cols,
15     redact_cols=DEFAULT_POLICY.redact_cols,
16     blank_cols={
17         "accounts": {"profile_image_url", "location"},
18         "posts": {"location"},
19     },
20 )
21
22 # -- Opt-in to Presidio-based PII detection on post bodies and bios ----
23 pii_policy = PiiPolicy(rules={
24     "accounts": {"bio": {
25         "PERSON": PiiRule(PiiAction.REPLACE),
26         "PHONE_NUMBER": PiiRule(PiiAction.REPLACE),
27         "EMAIL_ADDRESS": PiiRule(PiiAction.REPLACE, replacement="[EMAIL]"),
28         "CREDIT_CARD": PiiRule(PiiAction.REPLACE),
29         "URL": PiiRule(PiiAction.REPLACE,
30             replacement=lambda u: "URL_" + u.split("/") [2]
31             if "/" in u else u),
32     }},
33     "posts": {"body": {
34         "PERSON": PiiRule(PiiAction.REPLACE),
35         "PHONE_NUMBER": PiiRule(PiiAction.REPLACE),
36         "CREDIT_CARD": PiiRule(PiiAction.REPLACE),
37         "MENTION": PiiRule(PiiAction.HASH),
38         "HASHTAG": PiiRule(PiiAction.REPLACE,
39             replacement=lambda t: t.lower().lstrip("#")),
40     }},
41 })
42
43 cfg = PseudonymizeConfig(
44     src_db_name = "election2023_raw",
45     dst_db_name = "election2023_pseudo",
46     pepper = pseudo_vars.pepper,
47     algorithm = Algorithm.SHA256,
48     chunk_rows = 5_000,
49     time_window = ("2023-05-01", "2023-05-15"),
50     num_workers = 4,
51     ask_reinit = True,
52     pii_policy = pii_policy,
53     nlp_configuration = {
54         "nlp_engine_name": "spacy",
55         "models": [{"lang_code": "en", "model_name": "en_core_web_lg"}],
56     },
57 )
58
59 Pseudonymizer(cfg, custom_policy).run()

```

Listing 3: **Pseudonymization.** A fully configured pseudonymization run: custom column policy, Presidio-based PII detection with per-entity-type rules, a time window, and parallel workers.

Sample Deletion Request Code Snippet

```

1  from smdt.config import PseudonymizationVariables
2  from smdt.pseudonymizer import Eraser, ErasureTarget, ErasureMode
3
4  pseudo_vars = PseudonymizationVariables()
5  eraser = Eraser(
6      targets=[
7          ErasureTarget(db_name="my_local_db", mode=ErasureMode.DELETE, is_pseudonymized=False
8          ),
9          ErasureTarget(db_name="my_local_db_pseudo", mode=ErasureMode.SCRUB, is_pseudonymized
10         =True),
11         ],
12         pepper=pseudo_vars.pepper,
13     )
14  report = eraser.erase("real_account_id_123", identity_column="account_id")

```

Listing 4: **Deletion Request.** A sample code for processing a data deletion request across the raw and pseudonymized databases, using hard deletion on the source database and in place scrubbing on the released dataset.

Sample MultiStore Query

```

1  from smdt.multistore import MultiStore
2
3  with MultiStore() as ms:
4      ms.attach("twitter", db_name="twitter_db")
5      ms.attach("bluesky", db_name="bluesky_db")
6
7      # Each attached dataset is exposed as a schema named after its alias --
8      # ordinary SQL joins/unions work across them because every SMDT
9      # dataset shares the same standardized schema.
10     df = ms.query("""
11         SELECT tw.username,
12                tw.follower_count AS tw_followers,
13                bs.follower_count AS bs_followers
14     FROM twitter.accounts tw
15     JOIN bluesky.accounts bs
16         ON lower(tw.username) = lower(bs.username)
17     """)

```

Listing 5: **Cross-Dataset Query** A sample code for doing a multi store query for cross dataset analysis.

3.4. Enrichment modules

Raw social media data often lacks the depth required for comprehensive research, making it necessary to enrich the foundational data with additional context about accounts and content. For example, complex tasks such as detecting coordinated activities and tracking online campaigns rely heavily on this supplementary information. To support this, our framework incorporates external tools and machine learning models designed for specific tasks, treating the information obtained from these additional models as an enrichment of the collected data.

Examples of data enrichment includes BotometerLite for social bot detection [38] and M3Inference model for estimating user demographics [39]. One can consider additional models to infer account-level details like inferring location from profile descriptions, fake personas from profile pictures, or detecting anomalous followers from social network topology [40].

Social media content also offers valuable information and models for detecting sentiment, hate speech or toxicity can be applied to textual information [41,42]. More sophisticated models can incorporate links shared with the tweets to estimate information credibility.

Images or videos attached to the content can be also processed with multi-model approaches of fake news detection.

All these account- and content-level enrichment can be used to study information cascades and coordination among accounts. We offer few analysis modules that process base information and enrichment; but we want to emphasize SMDT is highly flexible to extend with other existing models and users' own methodologies.

3.4.1. Social networks module

The Social Networks Module is designed to construct tailored networks for analyzing information diffusion. It enables users to generate three primary network architectures:

- **User–User Interaction Networks:** Mapping direct interactions between individuals (e.g., SHARE and QUOTE networks).
- **Co-occurrence Networks:** Identifying the shared usage of specific features, such as hashtags and mentions.
- **Bipartite Networks:** Modeling relationships between two distinct entity types (e.g., accounts and shared hashtags or domains).

Additionally, the module supports temporal network construction, allowing users to analyze structural evolution over time and track the

dynamic spread of information. Listing 6 demonstrates how to build temporal share networks using a standardized dataset.

Share network generation

```

1  from datetime import datetime, timedelta
2  from smdt.store.standard_db import
   StandardDB
3  from smdt import networks
4
5  db = StandardDB("my_local_db")
6
7  windows = networks.
   user_interaction_over_time(
8      db,
9      interaction="SHARE",
10     start_time=datetime(2023, 5, 14),
11     end_time=datetime(2023, 5, 15),
12     step=timedelta(hours=1),
13     weighting="count",
14     min_weight=3,
15 )
16
17 for win in windows:
18     ws = win["window_start"]
19     we = win["window_end"]
20     net = win["network"]
21     print(ws, we, net.meta["edge_count"])

```

Listing 6: **Network module** Temporal SHARE (Retweet) network generation.

3.4.2. Natural language processing module

We provide an advanced enrichment layer for standardized datasets, allowing users to apply both online and offline NLP models directly to their data. The offline sublayer enables the local execution of Hugging Face models using standard model identifiers, while the online sublayer provides a unified interface for external providers such as Gemini, Claude, and OpenAI, alongside locally hosted vLLM and Ollama servers. This flexible architecture allows users to deploy arbitrary models, from sentiment and hate speech classifiers to custom, prompt-specific generative tasks.

Before any enrichment model receives a batch of rows, a configurable privacy preprocessor applies the same redaction pipeline used by the pseudonymizer. Mentions, emails, and URLs in designated columns are scrubbed prior to transmission, and when the optional PII detection layer is enabled, phone numbers, personal names, and credit card numbers are additionally replaced with entity-type placeholders. This design ensures that even when enrichment calls external LLM APIs, no raw personal identifiers leave the local environment. For fully offline operation, locally hosted models via vLLM or Ollama avoid external transmission entirely. Listing 7 demonstrates a fully configured enrichment run using Anthropic's Claude model, with the privacy preprocessor enabled to redact PII in the post body before any text is sent to the external API.

3.4.3. Bot detection

Behavioral anomalies and malicious intentions are common for all platform. However, their strategies and mechanisms may change. In literature, there are models trained on labeled datasets for particular platforms like Twitter. Botometer is one of the well-known model for Twitter [43], but this model extracts platform specific features to capture diverse automated behavior. Because of that this model will be only usable for Twitter specific datasets upon the availability of the profile level information.

4. Performance and scalability evaluation

To assess the performance and scalability of the SMDT, we evaluate the ingestion pipeline, pseudonymization module, and query performance against a real dataset of 10 million posts from Twitter's January to February 2023 Turkish election dataset [27], totaling approximately 86 million normalized database rows: 10.7M posts, 64.0M entities, 10.5M actions, and 815K accounts. All benchmarks run on a single machine with the dataset and Postgres instance co-located. Fig. 2 summarizes the three main dimensions of the evaluation. The ingestion benchmark feeds the pipeline incrementally at five cumulative checkpoints of 10^3 , 10^4 , 10^5 , 10^6 , and 10^7 records, with pre-sliced input files capped at 12,500 records each to provide file-level parallelism at the 100K+ checkpoints. At each checkpoint we record both cumulative state, including database size, per-table row counts, and TimescaleDB chunk counts, and incremental performance for that stage alone, including ingestion throughput, per-table flush rates, and process memory sampled continuously throughout the run. This allows us to distinguish one-time, amortizing costs such as partition and chunk creation from steady-state behavior.

The pipeline was configured with a batch size of 10^5 records and a database insertion chunk size of 10^5 , with an on-conflict policy of DO NOTHING applied uniformly across every model table to silently skip duplicate-key rows during insertion. To evaluate the effect of parallelism, we repeated the full cumulative benchmark twice, under identical input and configuration except for one variable: a single-threaded baseline with `num_workers` set to 1, matching the pipeline's original execution model, and a parallel configuration with `num_workers` set to 8 using a process pool to ingest multiple files concurrently, each into its own dedicated database, so the two runs never interfere and can be compared directly at each checkpoint.

Ingestion throughput. Fig. 2A shows ingestion throughput as a function of cumulative dataset size. Single-threaded throughput remains flat across four orders of magnitude, stabilizing at approximately 360 records per second regardless of database size. The growing hypertable chunk count, reaching approximately 2500 chunks for posts at 10M records, does not measurably degrade write throughput, validating the TimescaleDB chunk-append model for append-mostly workloads. Parallel execution provides no benefit below 100K records, where each checkpoint consists of a single input file and cannot distribute work across workers. At 100K records and beyond, where multiple files per checkpoint enable file-level parallelism, the speedup grows with scale: 4.4× at 100K, 5.5× at 1M, and 5.9× at 10M records, or 2092 rec/s versus 356 rec/s. The speedup does not reach the theoretical maximum of 8×, consistent with the serial fraction of the pipeline: standardization, deduplication, and database writes all remain single-threaded within each worker batch. The 10M-record ingestion completes in approximately 1.2 h with 8 workers versus 7.0 h with a single worker.

Memory scaling. Process memory is sampled every two seconds throughout each benchmark run by summing the resident set size of the main process and all recursive child processes. Single-threaded memory grows from 55 MB at 1K records to 462 MB at 10M records, reflecting buffered batches and the growing pipeline state. The growth is sub-linear: a 10,000× increase in data volume produces only an 8.4× increase in memory. Parallel execution with 8 workers uses approximately 3× more memory, peaking at 1376 MB at 10M records, consistent with each worker independently buffering batches for its assigned files. The main dispatcher process itself uses only approximately 45 MB. The 3× memory multiplier for a 5.9× throughput gain represents a favorable memory-performance tradeoff.

Database storage. The final single-threaded database occupies approximately 44.6 GB for 86 million rows. Posts and entities dominate storage volume: entities alone account for 64 million rows, as each post generates an average of 6 extracted entities for hashtags, mentions, URLs, and media. The database grows roughly linearly with input record count: 44 MB at 1K, 71 MB at 10K, 310 MB at 100K, and

TextGen Enricher with Privacy Layer

```

1  import os
2  from smdt.store.standard_db import StandardDB
3  from smdt.enrichers.runner import run_enricher
4  from smdt.enrichers.llm_textgen import TextGenConfig
5  from smdt.pseudonymizer.pii_policy import DEFAULT_PII_POLICY
6
7  db = StandardDB(db_name="my_local_db")
8
9  config = TextGenConfig.for_anthropic(
10     model="claude-3-5-sonnet-20241022",
11     api_key=os.environ["ANTHROPIC_API_KEY"],
12     model_id_postfix="v1_sentiment_claude",
13     # Prompt configuration
14     system_prompt="You are a helpful assistant.",
15     user_template="Analyze the sentiment of this post: {body}",
16     # Execution settings
17     only_missing=True,
18     batch_size=10,
19     reset_cache=True,
20     max_tokens=1000,
21     # Privacy layer: mentions, emails, phone numbers, and names in
22     # "body" are redacted or hashed before the row leaves the machine.
23     privacy_fields=["body"],
24     pii_policy=DEFAULT_PII_POLICY,
25     pepper=os.environ["PEPPER"].encode(),
26 )
27
28 run_enricher("textgen", db=db, config=config)

```

Listing 7: **Text-level enrichment.** TextGen enricher configuration using Anthropic's Claude model, with the built-in privacy preprocessor enabled to redact PII in body before transmission to the external API.

approximately 2.8 GB at 1M records. The entities table is the primary driver of row count and storage volume, consistent with the extraction of multiple entity rows per post during standardization.

Query performance. Fig. 2B reports query latency across 13 representative query types against the final 10M-record single-threaded database, covering point lookups, composite index range scans, hypertable chunk-exclusion aggregations, spatial queries, a two-table join, and one deliberately unindexed text search as a baseline. Each query is repeated 20 times with 5 distinct real parameter values sampled from the database, producing 100 warm-cache latency measurements per query. Blue bars show the median latency (p50); red tick marks show the 95th percentile (p95). Point lookups against non-hypertable tables return in 3.1 ms at p50 for accounts, which probes 42 hypertable chunks. Point lookups against the posts hypertable, which has approximately 2500 chunks at this scale, return in 13.3 ms at p50, reflecting the overhead of appending across all chunks. Composite index range scans perform comparably: retrieving an account's timeline and fetching a conversation thread both complete in 13 to 14 ms at p50. A two-table join between entities and posts, filtered by account and entity type, completes in 15.8 ms at p50 and 279.8 ms at p95; the high p95 reflects cases where the sampled account has a large number of hashtag entities to join against. Hypertable chunk exclusion benefits are visible in time-filtered queries: counting posts in a 7-day window completes in 132 ms at p50, while bucketing posts per day over a 30-day window completes in 273 ms at p50. The top-hashtags aggregation, which scans the space-partitioned entities hypertable, is the most expensive indexed query at 583 ms at p50 and 811 ms at p95. The entities table benefits from a space dimension on entity type in addition to the time dimension, so a query filtered to HASHTAG scans only the relevant partitions. As a baseline for the value of indexing, an unindexed case-insensitive substring search on post bodies requires a full sequential scan across all post chunks and completes in 2623 ms

at p50, approximately 200× slower than an indexed point lookup. A spatial query finding posts within 50 km of a point returns in 71 ms at p50 using the GIST index on the PostGIS geometry column, though this query is data-dependent on the density of geotagged posts in the dataset.

Pseudonymization throughput. Fig. 2C reports the parallel speedup of the pseudonymization pipeline per table. The pipeline transforms the complete 86M-row database from plaintext to pseudonymized form. Because the pipeline reads from one database and writes to another, its profile is split between CPU-bound transformation, hashing identifiers and redacting free text, and I/O-bound database operations: reading source rows via server-side cursors and flushing transformed batches. We benchmark both serial and eight-worker parallel execution, where only the transformation step is parallelized; database reads and writes remain in the main process. End-to-end pseudonymization of all 86 million rows completes in 8.7 h with a single worker and 2.7 h with 8 workers, an overall speedup of 3.2×. However, the speedup is highly table-dependent. The posts table, with 10.7M rows dominated by regex redaction of body text, achieves a 6.4× speedup, reflecting the CPU-bound nature of text processing. The entities table, with 64.0M rows dominated by lightweight entity-type-aware sanitization, achieves 1.9×. The actions table, with 10.5M rows and no free-text columns, involves very fast per-row hashing of identifier columns and is actually 9% slower with 8 workers, since the per-row transformation is so cheap that the inter-process communication overhead exceeds the compute savings. This result demonstrates that parallelization should be applied selectively, or that the transform chunk size should be tuned per table, rather than uniformly enabled.

Compression. TimescaleDB supports per-chunk columnar compression that reorganizes data within each hypertable chunk into a column-oriented layout and applies type-specific encoding: delta-of-delta encoding for timestamps, dictionary encoding for low-cardinality columns

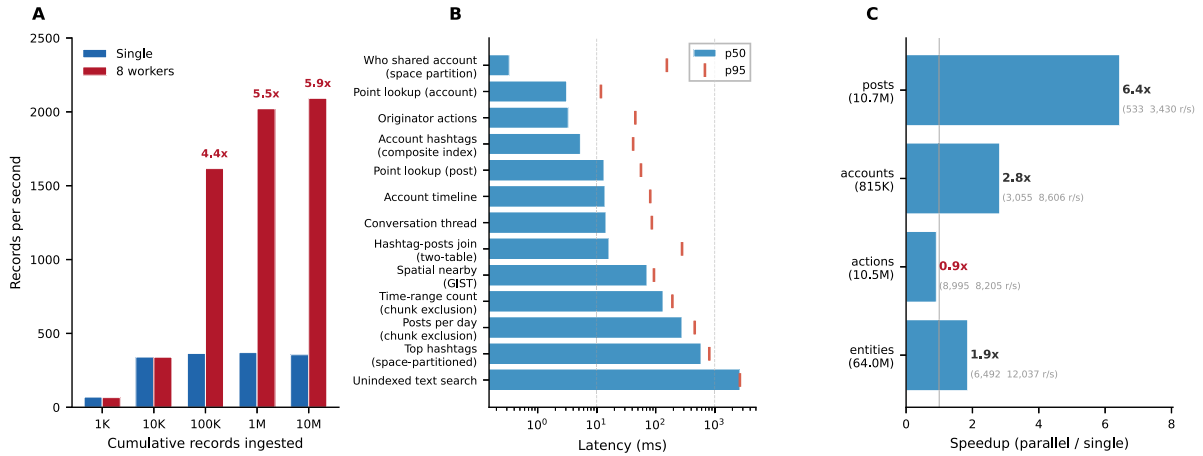


Fig. 2. Performance and scalability summary. (A) Ingestion throughput at five cumulative checkpoints comparing single-threaded and eight-worker parallel execution. Speedup annotations shown for checkpoints with file-level parallelism. (B) Query latency on the 10M-record database, ordered by median. Blue bars show the 50th percentile (p50); red tick marks show the 95th percentile (p95). (C) Pseudonymization speedup per table with eight workers. The dashed line at 1.0x marks equal performance.

such as platform and entity type, and LZ4 or zstd compression for text and JSONB columns. Typical space savings for social media datasets range from 90% to 97%. Because compressed chunks cannot accept new inserts, compression policies in the schema are deliberately left disabled during bulk ingestion and are intended to be activated afterward on a per-database basis via `add_compression_policy`. Under this design, chunks older than a configurable interval, typically 7 days, are automatically compressed while recent chunks remain uncompressed for ongoing writes. Once enabled, aggregation and range-scan queries benefit from scanning fewer disk pages: a chunk-exclusion count query that reads several hundred thousand shared buffer blocks uncompressed may read an order of magnitude fewer blocks from compressed chunks. Point lookups experience a negligible CPU overhead from on-the-fly decompression and remain within a few milliseconds of their uncompressed latency. For the benchmark database reported here, enabling compression would reduce the 44.6 GB storage footprint to an estimated 3 to 5 GB and would lower the latency of the top-hashtags and posts-per-day aggregation queries by a factor of 2 to 5x, while leaving indexed point-lookup latency essentially unchanged.

Machine specification. All benchmarks were run on a server with two Intel Xeon Gold 6152 processors (44 physical cores, 88 threads, 2.10 GHz base, 3.70 GHz turbo), 125 GB of DDR4 RAM, and network-attached storage over NFS. Postgres 14.19 was configured with `shared_buffers` set to 32 GB and `effective_cache_size` set to 96 GB, with the TimescaleDB and PostGIS extensions enabled. The dataset and database both resided on the same NFS mount, so read and write I/O during ingestion shared the same storage path. Parallel ingestion used 8 workers, which is substantially fewer than the available core count; the parallelism ceiling is determined by the number of pre-sliced input files per checkpoint rather than by CPU availability, and the server had sufficient headroom for all 8 workers plus the Postgres backends to run without contention.

5. Case studies

In this section, we demonstrate the versatility of SMDT through a series of selected case studies. Each subsequent subsection utilizes distinct configurations to showcase the specific functionalities and performance of our system. Datasets analyzed in these studies comes from different APIs, or format due to scraping; however, SMDT standardize these data and apply the same tools for downstream tasks.

5.1. Comparative analysis of stance toward the Russia-Ukraine war

We selected the *TwitterArchive* and *TruthSocial* datasets due to their substantial political discourse regarding the Russia-Ukraine war. Following the standardization of the *USC-TruthSocial* and *USC-X* datasets, we filtered the posts using war-related keywords (listed in the Appendix). This process yielded approximately 58,000 posts from TruthSocial and roughly 1.74 million posts (thirty times the volume) from Twitter. To mitigate this significant volume disparity and ensure balanced comparisons, we constructed 30 bootstrapped samples from the Twitter dataset, each containing 58,000 posts. Next, we deployed the `deepseek-ai/deepseek-llm-7b-chat`⁴ model via the vLLM tool within our enrichment layer to identify each post's stance toward the conflict. The model was prompted to classify each post into one of five categories: Pro-Ukraine (expressing support for Ukraine, its government, military, or actions), Anti-Ukraine (criticizing or opposing Ukraine), Pro-Russia (supporting Russia, its government, military, or actions), Anti-Russia (criticizing or opposing Russia), or Neutral (showing no clear stance or not directly addressing the conflict). To validate this stance detection pipeline, a human annotator labeled 300 posts from each platform (600 total), upon which the pipeline achieved an F1-score of 0.62. Finally, to compare the ideological leanings across the two platforms, we conducted two-proportion Z-tests using the stance distributions from each of the 30 sampled batches. In every iteration, the p-values fell below the conventional significance threshold of 0.05. This indicates that TruthSocial consistently contained a significantly higher proportion of Russia-leaning posts than Twitter, a finding that holds true even after rigorously accounting for sampling variability.

5.2. Temporal sentiment analysis

We standardized four distinct social media datasets for our analysis: *USC-TruthSocial*, *USC-X*, the 2023 Turkish General Election dataset [27], and a 2023 Finnish Parliamentary Election dataset. To ensure linguistic accuracy across these diverse sources, we applied language-specific sentiment models. For the English datasets, we utilized `bartweet-base-sentiment-analysis`,⁵ optimized for informal syntax and emojis. For the Turkish dataset, we applied `TurkishBERTweet-Lora-SA`,⁶ a LoRA-fine-tuned transformer tailored for Turkish discourse [44]. Finally, for the Finnish dataset, we

⁴ <https://huggingface.co/deepseek-ai/deepseek-llm-7b-chat>.

⁵ huggingface.co/finiteautomata/bartweet-base-sentiment-analysis.

⁶ huggingface.co/VRLLab/TurkishBERTweet-Lora-SA.

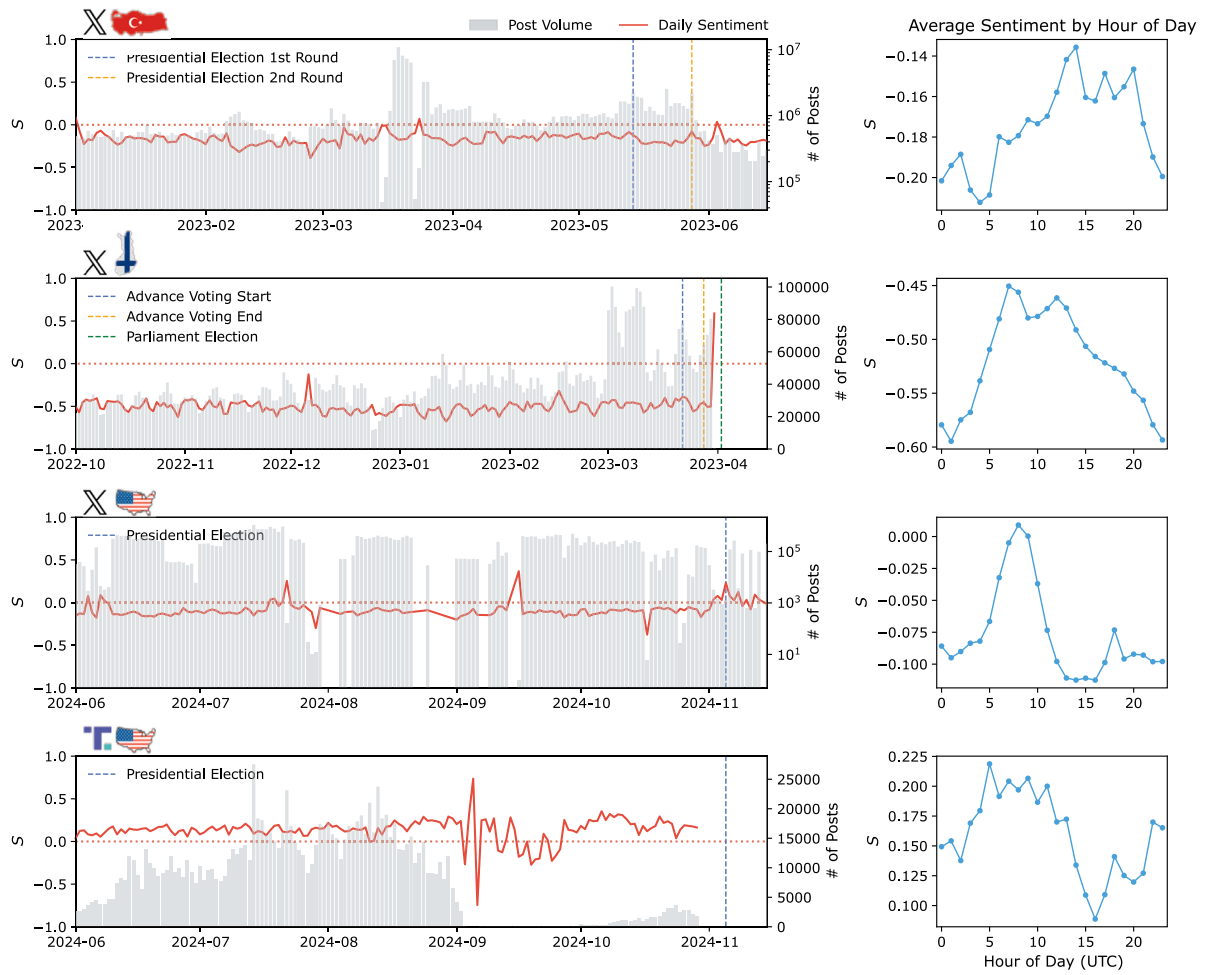


Fig. 3. Daily sentiment and post volumes. We selected election events and surrounding dates for analysis with their corresponding hourly sentiment trends. S indicates the average sentiment of posts.

employed `bert-finnish-sentiment-analysis`,⁷ fine-tuned for high precision in low-resource contexts.

Fig. 3 illustrates the temporal dynamics of daily sentiment and post volumes for each dataset, spanning from approximately four to six months prior to their respective elections. The left-hand panels display both aggregated daily sentiment and daily post volume. In the Turkish dataset, which begins in January 2023, generally we have aggregated negative sentiments for all of the days. Specially, there is a decline in the negative sentiment around February 2023 and increase in the number of posts; this reflects the impact of the regional earthquakes. Additionally, the drop observed in the March 2023 post volume data coincides with Twitter's structural migration from API v1 to API v2.

The Finnish dataset exhibits a distinct, cyclical posting frequency. This pattern shows a marked increase beginning in the first two weeks of March 2023, driven by the lead-up to advance voting. As a large portion of the Finnish electorate votes early, activity naturally peaked around the official advance voting period from March 22 to March 28. Meanwhile, the US datasets (*USC-X* and *USC-TruthSocial*) demonstrate more volatile posting patterns. This variance is an expected artifact of their compilation via web scraping, which introduces more noise compared to the API-based retrieval methods used for the Turkish and Finnish datasets. Furthermore, the *USC-TruthSocial* dataset contains a

lower total post volume than the others, further contributing to its structural variance.

The hourly plots reveal the intra-day variations of average sentiment for each dataset. When standardized to UTC, distinct patterns emerge reflecting geographic and platform-specific behaviors. Turkish Twitter sentiment peaks between approximately 10:00 and 15:00 UTC. Finnish Twitter, while maintaining a lower overall sentiment baseline, exhibits a peak during the morning hours. Comparing the two US platforms reveals diverging intra-day dynamics: *USC-X* sentiment reaches its maximum around 05:00 UTC before decreasing, whereas *USC-TruthSocial* sustains positive sentiment for a longer duration, featuring a distinct minimum at 16:00 UTC. These diurnal variations suggest that sentiment on social media corresponds closely to the daily behavioral cycles of its users.

5.3. Cross-platform hashtag and domain analysis

Following the standardization of the *TruthSocial* [13] and *Twitter-Archive* datasets, we extracted the hashtags and domains shared across both platforms. To calculate post-level toxicity scores, we utilized the Detoxify module [42] — which provides pre-trained models for toxic comment prediction — and subsequently aggregated these metrics by hashtag. The left panel of Fig. 4 illustrates the temporal alignment between the two platforms regarding hashtag usage and their respective toxicity differentials. Furthermore, the right panel of Fig. 4 presents the domain rankings alongside their corresponding Media Bias/Fact Check

⁷ huggingface.co/nisancoskun/bert-finnish-sentiment-analysis.

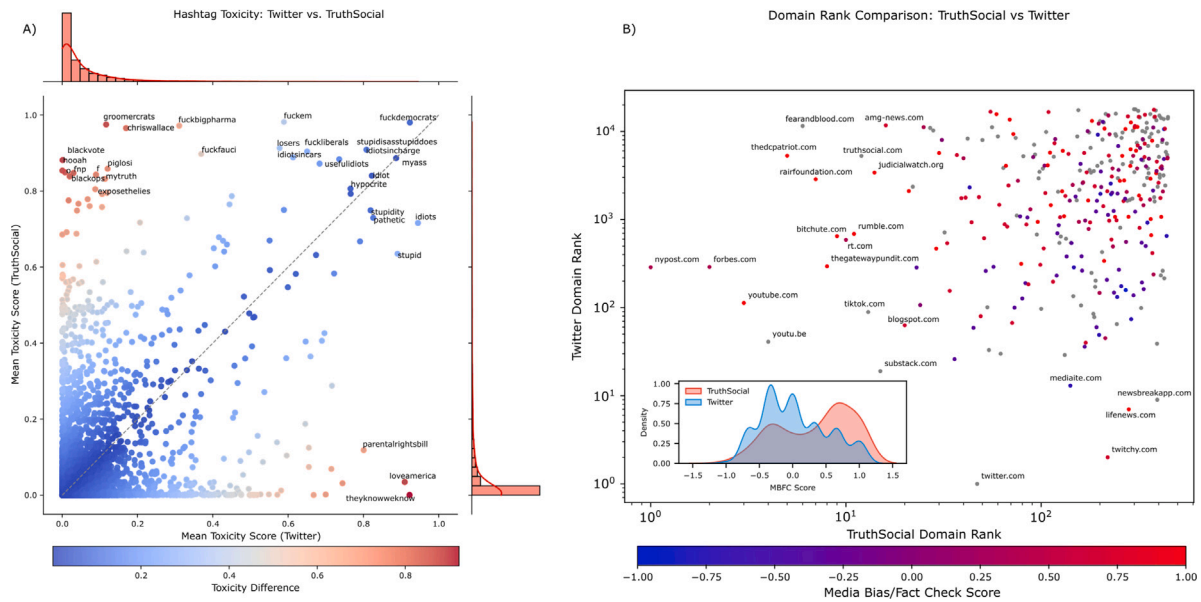


Fig. 4. Hashtag and domain analyses between Twitter and TruthSocial. (A) **Hashtag toxicity comparison.** Each point represents a hashtag appearing at least twice on both platforms. The x-axis shows the mean toxicity score on Twitter, and the y-axis shows the mean score on TruthSocial. Color indicates the absolute difference in toxicity between two platforms. (B) **Domain popularity and bias.** Each point represents a domain, positioned by its popularity rank on TruthSocial (x-axis) and Twitter (y-axis). Lower ranks correspond to higher popularity. Color reflects the Media Bias/Fact Check (MBFC) score, from left-leaning (blue) to right-leaning (red). The inset shows the distribution of MBFC scores per platform, illustrating a rightward shift in domain bias on TruthSocial. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

(MBFC) scores. These MBFC scores were sourced from prior work [45] and are publicly available via GitHub.⁸

The distribution of hashtag toxicity across both platforms reveals a significant concentration of low-toxicity content, as evidenced by the dense clustering of data points near the origin (0,0) and the strongly right-skewed marginal density distributions on the top and right axes. While the majority of hashtags remain relatively benign, the color gradient representing toxicity differences highlights notable platform-specific disparities. Specifically, a pronounced cluster in the upper-left region identifies hashtags that are highly toxic on TruthSocial (scores > 0.8) but yield very low toxicity on Twitter (scores < 0.2). These are frequently politically charged or targeted at specific individuals, such as #groomercrats and #piglosi. Conversely, a smaller subset of hashtags in the lower-right region demonstrates high toxicity on Twitter while remaining low on TruthSocial. Hashtags characterized by general profanity or broad insults tend to fall closer to the upper-right diagonal, indicating a shared level of high toxicity across both ecosystems.

This divergence in toxicity is accompanied by a stark ideological divide in shared media, characterized by polarized domain preferences across the platforms. As illustrated by the inset density plot, Twitter's domain distribution is bimodal, centering primarily within the negative-to-neutral range of the MBFC scores, which indicates a prevalence of left-leaning to moderate sources. In contrast, the distribution on TruthSocial is heavily concentrated in the positive range, signaling a dominant preference for right-leaning or conservative domains. This asymmetrical popularity is further evidenced in the scatter plot by a dense concentration of high-positive MBFC scores in the upper-left quadrant. High-ranking domains on TruthSocial, such as thegatewaypundit.com, bitchute.com, and rt.com, exhibit significantly lower visibility on Twitter. While mainstream outlets and traditional news platforms like youtube.com and forbes.com maintain high ranks across both ecosystems, left-leaning or centrist domains generally cluster toward the lower-right, demonstrating consistently stronger performance on Twitter than on TruthSocial.

5.4. Temporal hashtag co-occurrence analyses

We replicated the analysis of node clustering centrality versus degree as described by Lorenz-Spreen et al. [46], applying it to four datasets: Finnish and Turkish elections, USC-X, and USC-TruthSocial. To capture the temporal dynamics of the discourse, we constructed weekly hashtag co-occurrence networks, performed modularity-based community detection, and computed clustering centrality metrics. The results are illustrated in Fig. 5. As expected in hierarchical networks, low clustering coefficients characterize top-level hashtags associated with broad topics, while the upper part of the distribution comprises highly specific, niche hashtags.

In the Turkey Election dataset, high-level hashtags include election-centric terms such as #secim2023, #kilicdaroglu, and #chp, alongside #deprem (earthquake), which reflects the profound impact of the 2023 Earthquakes in south-east Turkey on the broader public discourse. Conversely, specific hashtags with high clustering in this dataset, such as #emeklikorucu, #herkesekadrovaruzm, and #11402lira, correlate with highly focused discussions regarding social security and economic issues. For the Finland Election dataset, broad, high-level hashtags are composed of general electoral terms like #vaalit2023, #eduskuntavaalit2023, and #eduskuntavaalit. In contrast, highly clustered, specific hashtags in this dataset highlight targeted topics and figures, including #rahaaon, #kiertotalousosaamista, #JariSarasvuo, and #Oikeisto. Similarly, the USC-X and USC-TruthSocial datasets feature broad, high-level hashtags related to the US elections, including #trump, #biden, and #maga. In contrast, the highly specific hashtags within these US datasets consist of targeted campaign or political markers, such as #uspolitics, #ninaturner2028, and #savingamerica.

These analysis demonstrates SMDT capabilities to parallelize analysis pipelines across multiple datasets without any additional effort and how it scales with large-datasets. This framework offers different modules to study content, accounts, and networks.

⁸ https://github.com/LazerLab/DomainDemo/blob/main/data/existing_labels/partisan_bias_scores.csv.

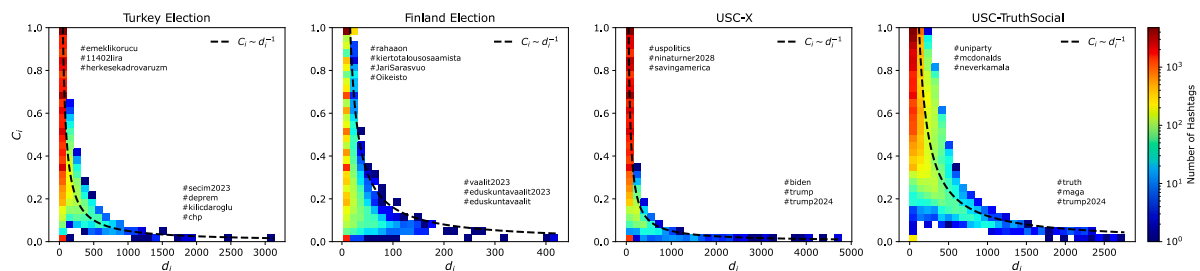


Fig. 5. Relationship between local clustering coefficient and node degree. Analysis conducted to investigate four different datasets. High-level hashtags for each context is identified and highlighted in figures.

6. Discussion and future work

To enable robust cross-platform comparisons, our framework maps diverse entity types to a uniform specification. This creates a shared analytical foundation that eliminates the structural and linguistic discrepancies often hindering collaborative research and data integration. By adhering to this standardized framework, researchers can ensure methodological consistency in their final evaluations, mitigating the risk of analytical errors stemming from data heterogeneity. Furthermore, our modular approach, which incorporates various user-level and content-level enrichment tools, enables researchers from diverse disciplinary backgrounds to seamlessly integrate these analyses into their workflows.

The necessity of such standardized analytical tools is underscored by recent regulatory shifts. Since 2018, the European Union has moved to regulate social media environments through frameworks such as the Digital Services Act (DSA). These regulations aim to combat the misuse of digital platforms, specifically addressing the drivers of online polarization, the dissemination of state-sponsored propaganda, and the systemic threats to global democratic processes. By providing a transparent and unified framework for data analysis, our work supports the overarching goals of these legislative efforts, offering a technical means to monitor and evaluate the digital ecosystem with greater precision.

While the current platform effectively supports the storage of statistical data, textual content, and file-based references for visual media, there are several avenues for future enhancement. A primary improvement involves integrating vector databases, such as ChromaDB [47], to support semantic enrichment and enable complex semantic queries. Although we currently store semantic features within enrichment tables in our standardized database, transitioning to a dedicated vector architecture would provide significantly greater flexibility and retrieval efficiency.

Additionally, driven by recent advancements in artificial intelligence and the rise of agentic models standardized under the Model Context Protocol (MCP) [48], our framework is well-positioned to serve as a critical bridge. By connecting distinct layers of social media data, it can reliably support the creation of advanced analytical agents capable of navigating and interpreting complex multi-platform environments.

Limitations While this work introduces a standardized infrastructure that enables researchers to conduct cross-platform analyses and allows non-technical users to leverage advanced enrichment tools, the current software is an early-stage Online Analytical Processing (OLAP) system. Future optimization is required to enhance its scalability and multi-dimensional flexibility. Additionally, the scope of cross-platform analysis is inherently constrained by external data availability. Consequently, we restricted our evaluation to the aforementioned case studies to demonstrate the system's baseline usability.

CRedit authorship contribution statement

Ali Najafi: Writing – original draft, Visualization, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Letizia Iannucci:** Writing – review & editing, Validation, Software. **Mikko Kivelä:**

Writing – review & editing, Supervision, Funding acquisition. **Onur Varol:** Writing – original draft, Visualization, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Co-author (Onur Varol) serves as an area editor for the OSNEM journal. Given his role as area editor, had no involvement in the peer review of this article and had no access to information regarding its peer review. Full responsibility for the editorial process for this article was delegated to another journal editor. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to acknowledge the support of CHIST-ERA for the CON-NET project and their local funding agencies for this grant. The agencies are the Scientific and Technological Research Council of Türkiye (grant: 222N311), the Research Council of Finland (grant: 357743).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.osnem.2026.100356>.

Data availability

No data was used for the research described in the article.

References

- [1] A. Hermida, F. Fletcher, D. Korell, D. Logan, Share, like, recommend: Decoding the social media news consumer, *Journal. Stud.* 13 (5–6) (2012) 815–824.
- [2] A.S. Kümpel, V. Karnowski, T. Keyling, News sharing in social media: A review of current research on news sharing users, content, and networks, *Soc. Media+ Soc.* 1 (2) (2015) 2056305115610141.
- [3] J. Chen, Y. Wang, et al., Social media use for health purposes: systematic review, *J. Med. Internet Res.* 23 (5) (2021) e17917.
- [4] J. Cole, C. Watkins, D. Kleine, Health advice from internet discussion forums: how bad is dangerous? *J. Med. Internet Res.* 18 (1) (2016) e4.
- [5] S. Kedvarin, K. Saengchote, Social media influencers: Evidence from YouTube and cryptocurrencies, 2023, Available At SSRN 4594081.
- [6] T. Renault, Market manipulation and suspicious stock recommendations on social media, 2017, Available At SSRN 3010850.
- [7] D. Freelon, Computational research in the post-API age, *Political Commun.* 35 (4) (2018) 665–668.
- [8] A. Bruns, After the 'APIcalypse': Social media platforms and their fight against critical scholarly research, in: *Disinformation and Data Lockdown on Social Platforms*, 2021, pp. 14–36.

- [9] R. Abbott, A. Botev, D. Boyda, D.C. Hackett, G. Kanwar, S. Racanière, D.J. Rezende, F. Romero-López, P.E. Shanahan, J.M. Urban, Applications of flow models to the generation of correlated lattice QCD ensembles, *Phys. Rev. D* 109 (9) (2024) 094514.
- [10] K. Kinder-Kurlanda, K. Weller, W. Zenk-Möltgen, J. Pfeffer, F. Morstatter, Archiving information from geotagged tweets to promote reproducibility and comparability in social media research, *Big Data Soc.* 4 (2) (2017) 2053951717736336, <http://dx.doi.org/10.1177/2053951717736336>.
- [11] O.C. Seckin, F.N. Silva, B.T. Truong, S. Kim, F. Huang, N. Liu, A. Flammini, F. Menczer, The rise of bluesky, 2025, arXiv preprint [arXiv:2504.12902](https://arxiv.org/abs/2504.12902).
- [12] A. Mekacher, M. Falkenberg, A. Baronchelli, The Koo dataset: An Indian microblogging platform with global ambitions, in: *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 18, 2024, pp. 1991–2002.
- [13] P. Gerard, N. Botzer, T. Weninger, Truth social dataset, in: *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 17, 2023, pp. 1034–1040.
- [14] K. Shah, P. Gerard, L. Luceri, E. Ferrara, Unfiltered conversations: A dataset of 2024 US presidential election discourse on truth social, 2024, arXiv preprint [arXiv:2411.01330](https://arxiv.org/abs/2411.01330).
- [15] S.J. Baele, L. Brace, T.G. Coan, Variations on a theme? Comparing 4chan, 8kun, and other chans' far-right "/pol/" boards, *Perspect. Terror.* 15 (1) (2021) 65–80.
- [16] E. Chandrasekharan, U. Pavalanathan, A. Srinivasan, A. Glynn, J. Eisenstein, E. Gilbert, You can't stay here: The efficacy of reddit's 2015 ban examined through hate speech, *Proc. ACM Hum.-Comput. Interact.* 1 (CSCW) (2017) 1–22.
- [17] K. Bryanov, D. Vasina, Y. Pankova, V. Pakholkov, The other side of deplatforming: right-wing telegram in the wake of trump's Twitter ouster, in: *International Conference on Digital Transformation and Global Society*, Springer, 2021, pp. 417–428.
- [18] C. Buntain, M. Innes, T. Mitts, J. Shapiro, Cross-platform reactions to the post-January 6 deplatforming, *J. Quant. Descr.: Digit. Media* 3 (2023).
- [19] L.M. Aiello, K. Chung, M.D. Conover, E. Ferrara, A. Flammini, G.C. Fox, X. Gao, B. Gonçalves, P. Grabowicz, K. Hong, et al., OSOMe: The IUNI observatory on social media, *PeerJ Comput. Sci.* 2 (2016).
- [20] A. Gruzd, P. Mai, Z. Vahedi, Studying anti-social behaviour on reddit with communalitic, 2020.
- [21] M. Mitrović Dankulov, A. Tomašević, S. Maletić, M. Anđelković, A. Vranić, D. Cvetković, B. Stupovski, D. Vudragović, S. Major, A. Bogojević, Multi-platform aggregated dataset of online communities (MADOC), in: *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 19, 2025, pp. 2529–2538, <http://dx.doi.org/10.1609/icwsm.v19i1.35954>.
- [22] M.M. Dankulov, A. Tomašević, S. Maletić, M. Anđelković, A. Vranić, D. Cvetković, B. Stupovski, D. Vudragović, S. Major, A. Bogojević, Multi-platform aggregated dataset of online communities (MADOC), 2025, arXiv preprint [arXiv:2501.12886](https://arxiv.org/abs/2501.12886).
- [23] E. Summers, Documenting the Now, Twarc: A command line tool (and python library) for archiving Twitter JSON, 2025, <https://github.com/DocNow/twarc>. (v2.14.1, accessed 2026).
- [24] R. Ackland, T. Graham, SocialMediaLab: Tools for collecting social media data and generating networks for analysis, 2017, <https://github.com/vosonlab/SocialMediaLab>. R package, no longer actively maintained; renamed vosonSML.
- [25] F. Guo, X. Wang, Y. Xie, Z. Wang, J. Li, L. Wang, A survey of datasets for information diffusion tasks, 2024, URL: <https://arxiv.org/abs/2407.05161>, [arXiv:2407.05161](https://arxiv.org/abs/2407.05161).
- [26] K. Esmukov, et al., Geopy: Python geocoding toolbox, 2023, <https://geopy.readthedocs.io/en/stable/>.
- [27] A. Najafi, N. Mugurtay, Y. Zouzou, E. Demirci, S. Demirkiran, H.A. Karadeniz, O. Varol, First public dataset to study 2023 Turkish general election, *Sci. Rep.* 14 (1) (2024) 8794.
- [28] A. Failla, G. Rossetti, "I'm in the bluesky tonight": Insights from a year worth of social data, 2024, arXiv preprint [arXiv:2404.18984](https://arxiv.org/abs/2404.18984).
- [29] PushShift, Gab posts - 2016-08 to 2018-10, 2018, <https://academictorrents.com/details/064f2953e8b16a9b33119874aa0b1a907d857bc1>. (Accessed 24 February 2026).
- [30] A. Mekacher, A. Papasavva, "I can't keep it up." a dataset from the defunct voat. co news aggregator, in: *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 16, 2022, pp. 1302–1311.
- [31] J. Patel, P. Paudel, E. De Cristofaro, G. Stringhini, J. Blackburn, iDRAMA-scored-2024: A dataset of the scored social media platform from 2020 to 2023, in: *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 18, 2024, pp. 2014–2024.
- [32] J. Baumgartner, S. Zannettou, B. Keegan, M. Squire, J. Blackburn, The pushshift reddit dataset, in: *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 14, 2020, pp. 830–839.
- [33] A. Balasubramanian, V. Zou, H. Narayana, C. You, L. Luceri, E. Ferrara, A public dataset tracking social media discourse about the 2024 us presidential election on twitter/x, 2024, arXiv preprint [arXiv:2411.00376](https://arxiv.org/abs/2411.00376).
- [34] M. Aliapoulos, E. Bevensee, J. Blackburn, B. Bradlyn, E. De Cristofaro, G. Stringhini, S. Zannettou, An early look at the parler online social network, 2021, arXiv preprint [arXiv:2101.03820](https://arxiv.org/abs/2101.03820).
- [35] European Parliament and Council of the European Union, Regulation (EU) 2016/679 of the European parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing directive 95/46/EC (general data protection regulation), *Off. J. Eur. Union L* 119 (2016) 1–88, 4.5.2016, <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [36] Presidio Contributors, Presidio: Data Protection and De-identification SDK, Originally developed by Microsoft Research, Maintained by the Data Privacy Stack community, <https://github.com/data-privacy-stack/presidio>.
- [37] M. Honnibal, I. Montani, S. Van Landeghem, A. Boyd, spaCy: Industrial-strength natural language processing in python, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Association for Computational Linguistics, 2020, pp. 38–45, <http://dx.doi.org/10.18653/v1/2020.emnlp-demos.6>, URL: <https://spacy.io>.
- [38] K.-C. Yang, O. Varol, P.-M. Hui, F. Menczer, Scalable and generalizable social bot detection through data selection, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34, 2020, pp. 1096–1103.
- [39] Z. Wang, S. Hale, D.I. Adelani, P. Grabowicz, T. Hartman, F. Flöck, D. Jurgens, Demographic inference and representative population estimates from multilingual social media data, in: *The World Wide Web Conference*, 2019, pp. 2056–2067.
- [40] Y. Zouzou, O. Varol, Unsupervised detection of coordinated fake-follower campaigns on social media, *EPJ Data Sci.* 13 (1) (2024) 62.
- [41] J. Camacho-Collados, K. Rezaee, T. Riahi, A. Ushio, D. Loureiro, D. Antypas, J. Boisson, L. Espinosa-Anke, F. Liu, E. Martínez-Cámara, et al., TweetNLP: Cutting-edge natural language processing for social media, in: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Association for Computational Linguistics, Abu Dhabi, U.A.E., 2022.
- [42] L. Hanu, Unitary team, Detoxify, Github, 2020, <https://github.com/unitaryai/detoxify>.
- [43] O. Varol, E. Ferrara, C. Davis, F. Menczer, A. Flammini, Online human-bot interactions: Detection, estimation, and characterization, in: *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 11, 2017, pp. 280–289.
- [44] A. Najafi, O. Varol, Turkishbertweet: Fast and reliable large language model for social media analysis, *Expert Syst. Appl.* 255 (2024) 124737.
- [45] K.-C. Yang, P. Goel, A. Quintana-Mathé, L. Horgan, S.D. McCabe, N. Grinberg, K. Joseph, D. Lazer, DomainDemo: a dataset of domain-sharing activities among different demographic groups on Twitter, *Sci. Data* 12 (1) (2025) 1251.
- [46] P. Lorenz-Spreen, F. Wolf, J. Braun, G. Ghoshal, N. Djurdjevic Conrad, P. Hövel, Tracking online topics over time: understanding dynamic hashtag communities, *Comput. Soc. Netw.* 5 (1) (2018) 9.
- [47] Chroma Core, Chroma: The AI-native open-source embedding database, 2023, <https://github.com/chroma-core/chroma>. (Accessed 20 February 2026).
- [48] Model Context Protocol, What is the model context protocol (MCP)?, 2026, <https://modelcontextprotocol.io>. (Accessed 22 February 2026).